

# Лабораторная работа № 9.

## Linux-router, PAT, Port Forwarding и WireGuard VPN

**Тема:** Маршрутизация Linux, IPv4 forwarding, UFW, PAT, Port Forwarding и WireGuard

**Цель:** Настроить gw01 как Linux-маршрутизатор, реализовать выход LAN через PAT, опубликовать web01 через Port Forwarding и предоставить VPN-доступ к LAN через WireGuard.

**Дисциплина:** Основные сервисы Linux для предприятия

### Лабораторная работа № 9. Linux-router, PAT, Port Forwarding и WireGuard VPN

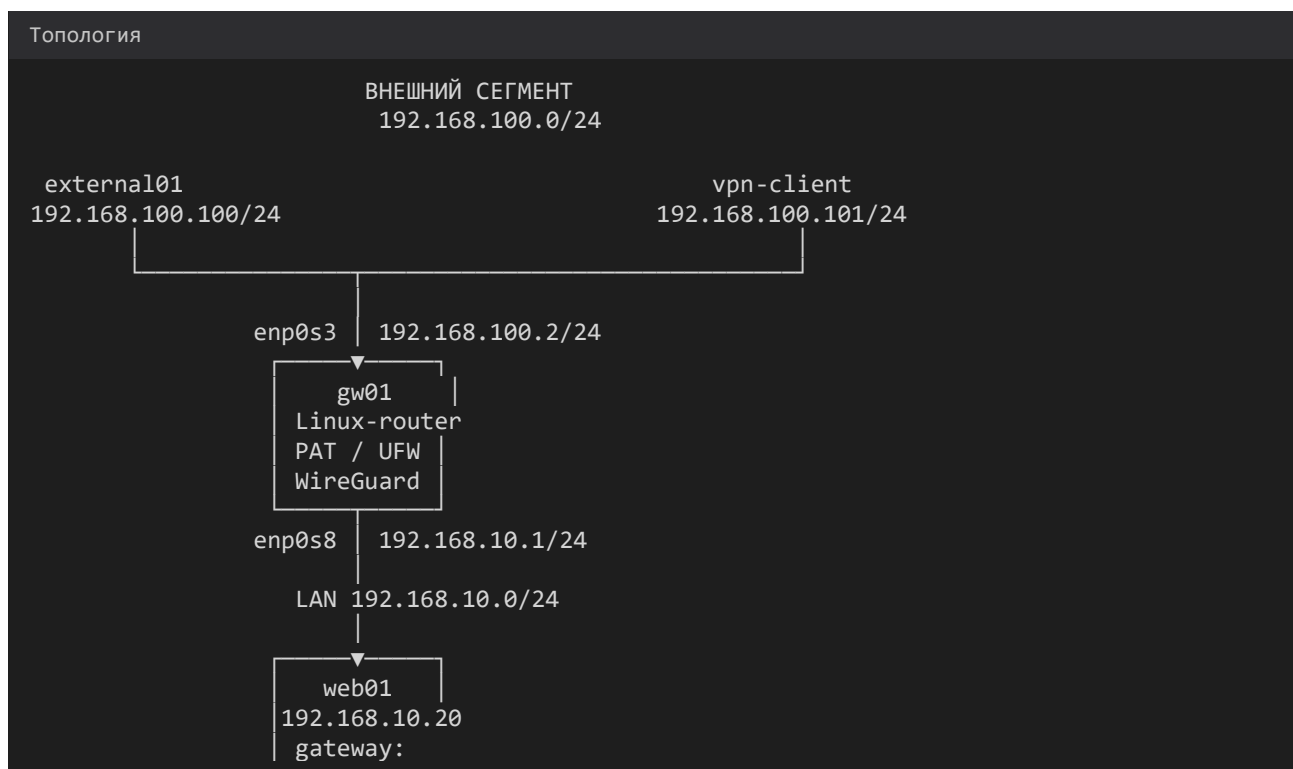
Цель работы — настроить Linux-сервер gw01 как маршрутизатор между внешним сегментом и внутренней LAN, включить IPv4 forwarding, организовать выход LAN наружу через PAT, опубликовать внутренний web-сервер через Port Forwarding и настроить удалённый доступ к LAN через WireGuard.

В исходном задании firewall настраивался другим инструментом. В этой версии лабораторной используется UFW, так как он уже изучался ранее. Отдельная теория firewall не повторяется: UFW применяется как знакомый инструмент для разрешения входящего и транзитного трафика.

Основное внимание в разделе NAT уделяется PAT — Port Address Translation. В учебном стенде внешний сегмент 192.168.100.0/24 не является реальным Интернетом: он только имитирует внешнюю сеть. Это позволяет безопасно проверить маршрутизацию, PAT, публикацию сервиса и VPN внутри VirtualBox.

После каждого этапа приведены контрольные команды и критерий перехода. Не переходите дальше, пока текущий этап не работает.

#### 1. Схема стенда



```
192.168.10.1
```

```
Port Forwarding:  
192.168.100.2:8080 → 192.168.10.20:80
```

```
WireGuard:  
gw01 wg0      = 10.8.0.1/24  
vpn-client wg0 = 10.8.0.2/32  
UDP 51820
```

## Адресный план

| Узел / интерфейс | Адрес              | Назначение                 |
|------------------|--------------------|----------------------------|
| gw01 enp0s3      | 192.168.100.2/24   | Внешний учебный сегмент    |
| gw01 enp0s8      | 192.168.10.1/24    | Gateway внутренней LAN     |
| web01            | 192.168.10.20/24   | Внутренний web-сервер      |
| web01 gateway    | 192.168.10.1       | Выход из LAN через gw01    |
| external01       | 192.168.100.100/24 | Проверка Port Forwarding   |
| vpn-client       | 192.168.100.101/24 | Удалённый WireGuard-клиент |
| gw01 wg0         | 10.8.0.1/24        | VPN-интерфейс сервера      |
| vpn-client wg0   | 10.8.0.2/32        | VPN-адрес клиента          |
| WireGuard        | UDP 51820          | Порт VPN                   |

## 2. Что должно получиться в конце

- gw01 имеет два обычных сетевых интерфейса и работает как маршрутизатор.
- net.ipv4.ip\_forward = 1 и сохраняется после перезагрузки.
- UFW разрешает только нужный входящий и routed-трафик.
- web01 использует 192.168.10.1 как default gateway.
- web01 может обращаться к external01 через gw01.
- Исходящий трафик LAN проходит через PAT/MASQUERADE.
- external01 открывает http://192.168.100.2:8080 и получает страницу web01:80.
- WireGuard поднимает туннель между vpn-client и gw01.
- Команда wg показывает latest handshake.
- vpn-client пингует 10.8.0.1 и достигает web01 192.168.10.20.
- Закрытые ключи WireGuard не попадают в отчёт.

## 3. Требования безопасности

- Перед изменением сетевых параметров и UFW сделайте снимок gw01.
- Работайте через консоль VirtualBox или сохраняйте резервный способ доступа к gw01.
- Перед включением UFW сначала добавьте необходимые правила управления.
- Port Forwarding настраивайте только для конкретного адреса и порта.

- Не публикуйте SSH и другие административные службы через Port Forwarding без необходимости.
- Private key WireGuard хранится только на своём узле.
- Файл `/etc/wireguard/wg0.conf` должен иметь права 600.
- В отчёт включайте только public key или замаскированные фрагменты конфигурации.

## Этап 1. Подготовка виртуальных машин и сетей VirtualBox

### 1.1. gw01

gw01 должен одновременно видеть внешний учебный сегмент и внутреннюю LAN. Поэтому ему нужны два сетевых адаптера.

1. Adapter 1 подключите к отдельной внешней сети VirtualBox, например Internal Network с именем wan-lab.
2. Adapter 2 подключите к Internal Network с именем lan-lab.
3. На внешнем интерфейсе назначьте 192.168.100.2/24.
4. На внутреннем интерфейсе назначьте 192.168.10.1/24.
5. Gateway на LAN-интерфейсе не задавайте.

Имена интерфейсов в задании `enp0s3` и `enp0s8` являются примером. На вашей VM они могут отличаться.

Определить реальные интерфейсы gw01

```
ip -br a
ip link
```

`ip -br a` показывает имя, состояние и IP-адрес каждого интерфейса. `ip link` полезен, если интерфейс ещё не имеет адреса. Запишите реальные имена WAN и LAN — они понадобятся в UFW.

### 1.2. web01

web01 находится только во внутренней LAN.

6. Подключите адаптер к Internal Network lan-lab.
7. Назначьте адрес 192.168.10.20/24.
8. Default gateway: 192.168.10.1.
9. DNS для основной части лабораторной не обязателен.

### 1.3. external01

external01 имитирует внешний клиент. Он должен находиться в той же внешней сети, что и `enp0s3` маршрутизатора.

10. Подключите адаптер к wan-lab.
11. Назначьте адрес 192.168.100.100/24.
12. Gateway для проверки Port Forwarding не требуется, так как gw01 находится в той же подсети.

### 1.4. vpn-client

vpn-client тоже находится во внешнем сегменте и имитирует удалённого пользователя.

13. Подключите адаптер к wan-lab.
14. Используйте адрес 192.168.100.101/24.
15. Этот адрес нужен только для доставки WireGuard-пакетов до 192.168.100.2:51820.

## 1.5. Пример /etc/network/interfaces на gw01

Если Debian использует ifupdown, конфигурация может выглядеть так. Перед копированием замените enp0s3/enp0s8 на реальные имена.

```
gw01: /etc/network/interfaces

auto lo
iface lo inet loopback

auto enp0s3
iface enp0s3 inet static
    address 192.168.100.2/24

auto enp0s8
iface enp0s8 inet static
    address 192.168.10.1/24
```

В этом учебном стенде gw01 соединяет две изолированные сети VirtualBox. Поэтому default gateway на gw01 для основных тестов не нужен.

## 1.6. Пример web01

```
web01: /etc/network/interfaces

auto lo
iface lo inet loopback

auto enp0s3
iface enp0s3 inet static
    address 192.168.10.20/24
    gateway 192.168.10.1
```

Именно строка gateway 192.168.10.1 заставляет web01 отправлять пакеты в другие сети через gw01.

## 1.7. Проверка адресов и маршрутов

```
На gw01

ip -br a
ip route
```

На gw01 должны быть одновременно сети 192.168.100.0/24 и 192.168.10.0/24.

```
На web01

ip -br a
ip route
```

На web01 должен быть маршрут default via 192.168.10.1.

```
На external01

ip -br a
ping -c 4 192.168.100.2
```

external01 должен видеть внешний интерфейс gw01.

```
На gw01 проверить web01  
ping -c 4 192.168.10.20
```

Критерий перехода: gw01 видит web01 и external01, а web01 использует gw01 как gateway.

## Этап 2. Включение IPv4 forwarding

По умолчанию Linux-сервер может принимать пакеты, предназначенные ему самому, но транзит между интерфейсами зависит от параметра `net.ipv4.ip_forward`.

### 2.1. Проверить текущее значение

```
На gw01  
sysctl net.ipv4.ip_forward
```

Если вывод заканчивается на `= 0`, маршрутизация IPv4 выключена.

### 2.2. Включить временно

```
Временное включение  
sudo sysctl -w net.ipv4.ip_forward=1
```

Команда изменяет параметр работающего ядра сразу, но после перезагрузки значение может вернуться к прежнему.

### 2.3. Сделать настройку постоянной

```
Создать sysctl-файл  
echo 'net.ipv4.ip_forward=1' | \  
sudo tee /etc/sysctl.d/99-router.conf
```

`tee` создаёт отдельный системный файл с параметром. Такой способ удобнее, чем редактировать общий `/etc/sysctl.conf`.

```
Применить постоянные параметры  
sudo sysctl --system
```

`sysctl --system` перечитывает `sysctl`-файлы и показывает применяемые значения.

### 2.4. Контрольная точка

```
Проверка  
sysctl net.ipv4.ip_forward
```

```
Ожидаемо
```

```
net.ipv4.ip_forward = 1
```

Критерий перехода: значение равно 1.

## Этап 3. Установка и базовая настройка UFW

UFW уже изучался ранее, поэтому здесь используем его как готовый инструмент. Цель — запретить ненужный входящий и транзитный трафик и явно разрешить необходимые сценарии.

### 3.1. Установка

```
На gw01
```

```
sudo apt update
sudo apt install ufw -y
```

Если ufw уже установлен, apt сообщит, что пакет имеет актуальную версию.

### 3.2. Сброс учебной конфигурации

Если на gw01 остались правила от предыдущих лабораторных, в отдельном учебном стенде их удобнее сбросить. Не делайте это на рабочем сервере.

```
Только для учебной VM
```

```
sudo ufw --force reset
```

reset удаляет текущие пользовательские правила UFW и возвращает базовую конфигурацию.

### 3.3. Политики по умолчанию

```
Базовые политики
```

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw default deny routed
```

deny incoming блокирует незапрошенные подключения к самому gw01. allow outgoing разрешает исходящие соединения gw01. deny routed запрещает транзит между интерфейсами до появления явных route-правил.

### 3.4. Если нужен SSH к gw01

Если вы управляете gw01 по SSH из LAN, добавьте правило ДО включения UFW.

```
Разрешить SSH только из LAN
```

```
sudo ufw allow from 192.168.10.0/24 to any port 22 proto tcp
```

Если используется только консоль VirtualBox, это правило необязательно.

### 3.5. Включить UFW

Включение

```
sudo ufw enable
```

Подтвердите включение, если UFW задаёт вопрос. После этого проверьте статус.

Проверка

```
sudo ufw status verbose
```

Ожидаются политики deny incoming, allow outgoing и deny routed.

## Этап 4. Настройка PAT для выхода LAN во внешний сегмент

Теперь web01 должен отправлять трафик в 192.168.100.0/24 через gw01. Основная идея — PAT: исходный адрес и при необходимости порт внутреннего клиента транслируются в адрес внешнего интерфейса gw01.

### 4.1. Разрешить routed-трафик LAN → WAN

На gw01

```
sudo ufw route allow in on enp0s8 out on enp0s3 from 192.168.10.0/24
```

ufw route используется именно для транзитного трафика, который проходит через gw01, а не адресован самому маршрутизатору.

in on enp0s8 означает вход из LAN, out on enp0s3 — выход во внешний сегмент. Если реальные имена интерфейсов другие, замените их.

### 4.2. Добавить PAT/MASQUERADE в UFW

UFW не имеет отдельной команды для MASQUERADE, поэтому NAT-блок добавляется в /etc/ufw/before.rules.

Открыть файл

```
sudo nano /etc/ufw/before.rules
```

В самое начало файла, ДО первой секции \*filter, добавьте блок:

PAT для LAN

```
*nat
:POSTROUTING ACCEPT [0:0]

-A POSTROUTING -s 192.168.10.0/24 -o enp0s3 -j MASQUERADE

COMMIT
```

Строка выбирает пакеты из сети 192.168.10.0/24, которые выходят через WAN-интерфейс, и выполняет MASQUERADE. В практическом смысле это PAT: внутренние соединения выходят от адреса gw01 и различаются состояниями и портами.

### 4.3. Перезапустить UFW

Применение NAT-правил

```
sudo ufw disable  
sudo ufw enable
```

Полное выключение и повторное включение гарантирует перечитывание before.rules.

Проверка UFW

```
sudo ufw status verbose
```

### 4.4. Проверка выхода web01

На web01

```
ip route  
ping -c 4 192.168.10.1  
ping -c 4 192.168.100.100
```

Первый ping проверяет gateway. Второй — прохождение транзитного трафика через gw01 во внешний сегмент.

Важно: ping показывает маршрутизацию и NAT, но не демонстрирует номера портов PAT, потому что ICMP не использует TCP/UDP-порты.

### 4.5. Наглядная проверка PAT через TCP

Чтобы увидеть работу соединения с портами, запустим временный HTTP-сервис на external01.

На external01

```
python3 -m http.server 8081 --bind 192.168.100.100
```

Команда запускает простой HTTP-сервер на TCP 8081. Терминал останется занят сервером — это нормально.

На web01 в другом терминале

```
curl http://192.168.100.100:8081/
```

Если получен HTML-ответ, TCP-соединение прошло через gw01.

### 4.6. Дополнительная проверка таблицы соединений

Для более наглядной демонстрации PAT можно установить conntrack на gw01.

На gw01

```
sudo apt install conntrack -y
```

Показать TCP-соединения

```
sudo conntrack -L -p tcp | grep 192.168.10.20
```

conntrack показывает исходный и ответный tuple соединения. Это позволяет увидеть, как ядро сопоставляет внутреннее соединение с трансляцией на внешней стороне.

Критерий перехода: web01 достигает external01 через gw01.

## Этап 5. Настройка web01 как HTTP-сервера

Для проверки Port Forwarding на web01 должна реально работать служба TCP 80.

### 5.1. Установка Nginx

На web01

```
sudo apt update
sudo apt install nginx -y
```

Проверить службу

```
systemctl status nginx --no-pager
```

Ожидается active (running).

### 5.2. Проверить локально

На web01

```
curl -I http://127.0.0.1
```

Ключ -I запрашивает только HTTP-заголовки. Ожидается HTTP/1.1 200 OK или другой успешный ответ Nginx.

### 5.3. Проверить с gw01

На gw01

```
curl -I http://192.168.10.20
```

Если gw01 получает ответ, сервис доступен внутри LAN и можно переходить к публикации.

## Этап 6. Port Forwarding 192.168.100.2:8080 → web01:80

Теперь внешний клиент должен обращаться к адресу gw01:8080, а маршрутизатор направит это соединение на web01:80.

## 6.1. Добавить DNAT в существующий NAT-блок

Не создавайте второй блок `*nat`. Откройте уже существующий `/etc/ufw/before.rules` и добавьте PREROUTING в тот же блок.

Итоговый NAT-блок

```
*nat
:PREROUTING ACCEPT [0:0]
:POSTROUTING ACCEPT [0:0]

-A PREROUTING -i enp0s3 -p tcp --dport 8080 -j DNAT --to-destination 192.168.10.20:80
-A POSTROUTING -s 192.168.10.0/24 -o enp0s3 -j MASQUERADE

COMMIT
```

PREROUTING обрабатывает входящий пакет до выбора конечного маршрута. DNAT меняет назначение с 192.168.100.2:8080 на 192.168.10.20:80.

## 6.2. Разрешить транзит к web01

На gw01

```
sudo ufw route allow in on enp0s3 out on enp0s8 \
to 192.168.10.20 port 80 proto tcp
```

DNAT только меняет адрес назначения. Это route-правило отдельно разрешает пройти транзитному пакету от внешнего интерфейса к web01:80.

## 6.3. Перезапустить UFW

Применить

```
sudo ufw disable
sudo ufw enable
sudo ufw status verbose
```

## 6.4. Проверить с external01

На external01

```
curl -I http://192.168.100.2:8080
```

external01 обращается не напрямую к web01, а к внешнему адресу gw01. Ожидается HTTP-ответ Nginx.

Критерий перехода: external01 получает ответ от внутреннего web01.

## 6.5. Если Port Forwarding не работает

Быстрая диагностика

| Проверка          | Команда                         |
|-------------------|---------------------------------|
| Работает ли Nginx | systemctl status nginx на web01 |

| Проверка                    | Команда                           |
|-----------------------------|-----------------------------------|
| Доступен ли web01 с gw01    | curl -I http://192.168.10.20      |
| Правильный ли gateway web01 | ip route на web01                 |
| Включён ли forwarding       | sysctl net.ipv4.ip_forward        |
| Есть ли UFW route rule      | sudo ufw status verbose           |
| Верный ли адрес external01  | ip -br a                          |
| Правильный ли внешний порт  | curl -I http://192.168.100.2:8080 |

## Этап 7. Установка WireGuard

Теперь настроим Remote Access VPN. vpn-client находится во внешнем сегменте, но после подключения должен получить доступ к внутренней LAN 192.168.10.0/24.

### 7.1. Установка на gw01

```
На gw01

sudo apt update
sudo apt install wireguard -y
```

Пакет устанавливает утилиты wg и wg-quick.

### 7.2. Установка на vpn-client

```
На vpn-client

sudo apt update
sudo apt install wireguard -y
```

## Этап 8. Генерация ключей WireGuard

Каждый реер имеет собственную пару private/public key. Закрытые ключи не передаются между узлами.

### 8.1. Ключи gw01

```
На gw01

sudo mkdir -p /etc/wireguard
sudo sh -c 'umask 077; wg genkey | tee /etc/wireguard/server_private.key | wg pubkey > /etc/wireguard/server_public.key'
```

umask 077 делает новые файлы доступными только владельцу. server\_private.key остаётся на gw01. server\_public.key можно передать клиенту.

```
Показать только public key

sudo cat /etc/wireguard/server_public.key
```

## 8.2. Ключи vpn-client

На vpn-client

```
sudo mkdir -p /etc/wireguard
sudo sh -c 'umask 077; wg genkey | tee /etc/wireguard/client_private.key | wg pubkey > /etc/wireguard/client_public.key'
```

Показать только public key

```
sudo cat /etc/wireguard/client_public.key
```

Запишите SERVER\_PUBLIC\_KEY и CLIENT\_PUBLIC\_KEY. Private key каждой стороны остаётся только на своём узле.

## Этап 9. Настройка WireGuard-сервера gw01

### 9.1. Получить private key сервера для конфигурации

Откройте файл только локально на gw01 и скопируйте значение в wg0.conf. Не помещайте его в отчёт.

На gw01

```
sudo cat /etc/wireguard/server_private.key
```

### 9.2. Создать wg0.conf

На gw01

```
sudo nano /etc/wireguard/wg0.conf
```

/etc/wireguard/wg0.conf

```
[Interface]
Address = 10.8.0.1/24
ListenPort = 51820
PrivateKey = <SERVER_PRIVATE_KEY>

[Peer]
PublicKey = <CLIENT_PUBLIC_KEY>
AllowedIPs = 10.8.0.2/32
```

Address — адрес gw01 внутри VPN. ListenPort — UDP-порт WireGuard. В Peer указывается public key клиента. AllowedIPs = 10.8.0.2/32 закрепляет VPN-адрес клиента за этим peer.

### 9.3. Защитить файл

Права

```
sudo chown root:root /etc/wireguard/wg0.conf
sudo chmod 600 /etc/wireguard/wg0.conf
```

wg0.conf содержит private key, поэтому обычные пользователи не должны иметь к нему доступа.

## Этап 10. UFW для WireGuard

### 10.1. Разрешить входящий UDP 51820

```
На gw01

sudo ufw allow in on enp0s3 proto udp to any port 51820
```

WireGuard использует UDP. Это правило разрешает VPN-пакеты только через внешний интерфейс.

### 10.2. Разрешить VPN → LAN

```
На gw01

sudo ufw route allow in on wg0 out on enp0s8 \
to 192.168.10.0/24
```

Это правило разрешает транзит от VPN-интерфейса к внутренней LAN.

### 10.3. Применить

```
Проверка

sudo ufw reload
sudo ufw status verbose
```

## Этап 11. Запуск WireGuard на gw01

```
Поднять wg0

sudo wg-quick up wg0
```

wg-quick читает /etc/wireguard/wg0.conf, создаёт интерфейс wg0 и применяет адреса.

```
Проверить интерфейс

ip -br a show wg0
```

Ожидается адрес 10.8.0.1/24.

```
Показать состояние WireGuard

sudo wg
```

До подключения клиента peer уже будет виден, но latest handshake может отсутствовать.

Автозапуск

```
sudo systemctl enable wg-quick@wg0
```

После перезагрузки systemd автоматически поднимет wg0.

## Этап 12. Настройка vpn-client

### 12.1. Создать конфигурацию

На vpn-client

```
sudo nano /etc/wireguard/wg0.conf
```

/etc/wireguard/wg0.conf

```
[Interface]
Address = 10.8.0.2/32
PrivateKey = <CLIENT_PRIVATE_KEY>

[Peer]
PublicKey = <SERVER_PUBLIC_KEY>
Endpoint = 192.168.100.2:51820
AllowedIPs = 10.8.0.0/24, 192.168.10.0/24
PersistentKeepalive = 25
```

Endpoint — физический адрес WireGuard-сервера во внешнем сегменте. AllowedIPs определяет, какие сети клиент отправляет в туннель. 192.168.10.0/24 даёт доступ к корпоративной LAN. 10.8.0.0/24 позволяет обращаться к VPN-адресам.

PersistentKeepalive = 25 полезен для клиентов за NAT и не мешает учебному стенду.

### 12.2. Защитить конфигурацию

На vpn-client

```
sudo chmod 600 /etc/wireguard/wg0.conf
```

### 12.3. Поднять туннель

На vpn-client

```
sudo wg-quick up wg0
```

Проверить

```
sudo wg
ip -br a show wg0
ip route
```

В таблице маршрутизации должен появиться маршрут к 192.168.10.0/24 через wg0.

## Этап 13. Проверка WireGuard

### 13.1. Проверить VPN-интерфейс сервера

```
На vpn-client  
  
ping -c 4 10.8.0.1
```

Если 10.8.0.1 отвечает, базовый туннель работает.

### 13.2. Проверить handshake

```
На gw01  
  
sudo wg
```

В секции peer должна появиться строка latest handshake с недавним временем. Счётчики transfer должны быть ненулевыми.

### 13.3. Проверить доступ к LAN

```
На vpn-client  
  
ping -c 4 192.168.10.20  
curl -I http://192.168.10.20
```

Ожидается ответ web01. Это доказывает, что трафик проходит из WireGuard в LAN через gw01.

Критерий перехода: handshake есть, 10.8.0.1 и 192.168.10.20 доступны.

## Этап 14. Проверка сохранения настроек после перезагрузки

UFW и WireGuard должны восстанавливаться автоматически, а ip\_forward — оставаться включённым.

### 14.1. Проверить автозапуск UFW

```
На gw01  
  
systemctl is-enabled ufw  
systemctl status ufw --no-pager
```

### 14.2. Проверить WireGuard

```
На gw01  
  
systemctl is-enabled wg-quick@wg0  
systemctl status wg-quick@wg0 --no-pager
```

### 14.3. Перезагрузка gw01

```
На gw01  
sudo reboot
```

После загрузки повторно выполните:

```
Контроль после reboot  
  
sysctl net.ipv4.ip_forward  
sudo ufw status verbose  
ip -br a show wg0  
sudo wg
```

После перезагрузки forwarding должен быть 1, UFW активен, wg0 поднят.

## Этап 15. Диагностический эксперимент

По исходному заданию нужно безопасно смоделировать одну ошибку и исправить её. Используем неверный AllowedIPs на vpn-client.

### 15.1. Создать ошибку

На vpn-client временно измените:

```
Неправильно  
  
AllowedIPs = 10.8.0.0/24
```

Теперь корпоративная сеть 192.168.10.0/24 больше не направляется в туннель.

```
Перезапустить клиентский туннель  
  
sudo wg-quick down wg0  
sudo wg-quick up wg0
```

```
Проверить  
  
ping -c 4 10.8.0.1  
ping -c 4 192.168.10.20  
ip route
```

Ожидаемый симптом: VPN-адрес сервера доступен, но web01 недоступен. Это показывает, что handshake сам по себе ещё не гарантирует правильную маршрутизацию к LAN.

### 15.2. Найти причину

```
Диагностика на vpn-client  
  
sudo wg  
ip route
```

wg показывает работающий peer, а ip route показывает отсутствие маршрута 192.168.10.0/24 через wg0.

### 15.3. Исправить

Верните:

Правильно

```
AllowedIPs = 10.8.0.0/24, 192.168.10.0/24
```

Применить

```
sudo wg-quick down wg0
sudo wg-quick up wg0
ping -c 4 192.168.10.20
```

В отчёте зафиксируйте симптом, команды диагностики, причину и исправление.

## 16. Универсальная диагностика

### Порядок проверки

| Симптом                            | Что проверить                       | Команды                                                  |
|------------------------------------|-------------------------------------|----------------------------------------------------------|
| web01 не видит gw01                | IP и LAN-интерфейсы                 | ip -br a; ping 192.168.10.1                              |
| web01 видит gw01, но не external01 | gateway, ip_forward, UFW route, PAT | ip route; sysctl net.ipv4.ip_forward; ufw status verbose |
| Port Forwarding не работает        | Nginx, DNAT, UFW route              | curl web01; curl gw01:8080; ufw status                   |
| Нет WireGuard handshake            | Endpoint, UDP 51820, ключи          | wg; ufw status; journalctl -u wg-quick@wg0               |
| Handshake есть, но LAN нет         | AllowedIPs, ip_forward, UFW route   | wg; ip route; sysctl; ufw status                         |
| После reboot VPN не поднялся       | systemd unit и конфигурация         | systemctl status wg-quick@wg0                            |

## 17. Итоговая проверка

- gw01 имеет 192.168.100.2/24 и 192.168.10.1/24.
- web01 имеет 192.168.10.20/24 и gateway 192.168.10.1.
- external01 имеет 192.168.100.100/24.
- vpn-client имеет 192.168.100.101/24.
- net.ipv4.ip\_forward = 1.
- UFW активен и routed-трафик разрешён только нужными правилами.
- PAT/MASQUERADE обеспечивает выход web01 во внешний сегмент.
- external01 получает HTTP-ответ через 192.168.100.2:8080.
- WireGuard слушает UDP 51820.
- wg показывает latest handshake.

- vpn-client пингует 10.8.0.1.
- vpn-client достигает 192.168.10.20.
- UFW, ip\_forward и WireGuard сохраняются после reboot.
- Диагностическая ошибка смоделирована и исправлена.

## 18. Паспорт маршрутизатора и VPN

Заполнить в отчёте

| Параметр             | Значение                               |
|----------------------|----------------------------------------|
| Имя маршрутизатора   | gw01                                   |
| Внешний интерфейс    | фактическое имя, например enp0s3       |
| Внешний адрес        | 192.168.100.2/24                       |
| Внутренний интерфейс | фактическое имя, например enp0s8       |
| Внутренний адрес     | 192.168.10.1/24                        |
| LAN                  | 192.168.10.0/24                        |
| net.ipv4.ip_forward  | 1                                      |
| PAT                  | 192.168.10.0/24 → MASQUERADE через WAN |
| Port Forwarding      | 192.168.100.2:8080 → 192.168.10.20:80  |
| web01                | 192.168.10.20                          |
| external01           | 192.168.100.100                        |
| WireGuard-интерфейс  | wg0                                    |
| VPN-сервер           | 10.8.0.1/24                            |
| VPN-клиент           | 10.8.0.2/32                            |
| WireGuard-порт       | 51820/udp                              |
| AllowedIPs клиента   | 10.8.0.0/24, 192.168.10.0/24           |
| Команда проверки     | sudo wg                                |

## 19. Что приложить к отчёту

16. Схему стенда: внешний сегмент, gw01, LAN, web01 и vpn-client.
17. ip -br а и ip route на gw01.
18. ip route на web01 с default via 192.168.10.1.
19. sysctl net.ipv4.ip\_forward.
20. sudo ufw status verbose.
21. Фрагмент NAT-блока /etc/ufw/before.rules без лишних системных данных.
22. Проверку доступа web01 → external01.
23. curl -I http://192.168.100.2:8080 на external01.
24. Фрагменты WireGuard-конфигураций без private key.
25. sudo wg с latest handshake и transfer.

- 26. ping 10.8.0.1 и ping/curl 192.168.10.20 с vpn-client.
- 27. Описание диагностической ошибки: симптом → диагностика → причина → исправление.

## 20. Контрольные вопросы

- 28. Зачем Linux-маршрутизатору net.ipv4.ip\_forward?
- 29. Чем локальный пакет отличается от транзитного?
- 30. Почему web01 нужен default gateway 192.168.10.1?
- 31. Что такое PAT и почему один внешний IP может обслуживать много внутренних соединений?
- 32. Чем PAT отличается от простого изменения только IP-адреса?
- 33. Зачем используется MASQUERADE?
- 34. Чем исходящий PAT отличается от Port Forwarding?
- 35. Что именно меняет DNAT при обращении к 192.168.100.2:8080?
- 36. Почему одного DNAT недостаточно без разрешённого routed-трафика?
- 37. Что такое peer в WireGuard?
- 38. Чем private key отличается от public key?
- 39. Что означает Endpoint в конфигурации WireGuard?
- 40. Что означает AllowedIPs на клиенте?
- 41. Что показывает latest handshake?
- 42. Почему WireGuard может иметь handshake, но не давать доступ к LAN?
- 43. Какие настройки должны пережить перезагрузку gw01?